Lecture 20 · Part V · Extended notes

Information retrieval: dense retrieval

Dataset: SciFact (BEIR), the same 5,183 abstracts

Why these notes exist

Lectures 19–22 sit outside Géron, the course's primary text. For those four lectures *these notes are the primary source*, and they are examined on exactly the same terms as the textbook parts. Everything in the slide deck for this lecture is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the corpus named above rather than a figure quoted from a paper.

Contents

Where we left off **2**

1 What an index of strings cannot do **3**

1.1 The three repairs that do not work . . . 3

2 Bi-encoders **3**

2.1 From tokens to one vector . . . 3

2.2 Why the vectors are normalised . . . 4

2.3 Encode both sides into one space . . . 4

2.3.1 One encoder or two? . . . 5

2.4 The whole retrieval system, in six lines . . . 5

2.5 The dimension is a dial too . . . 5

3 Training one **5**

3.1 What the objective must say . . . 6

3.2 The false-negative problem, sized . . . 6

3.3 What the bi-encoder gave up . . . 6

3.4 The operational cost the accuracy table never shows . . . 7

4 Against BM25, on the same judgements **7**

4.1 The protocol, stated before the numbers . . . 7

4.2 The result . . . 7

4.2.1 What this result does not license . . . 7

4.3 But look at recall@100 . . . 8

4.4 Do they find the same documents? . . . 8

4.5 The claim BM25 could not find . . . 8

5 Combining them, and paying for it **9**

5.1 Hybrid retrieval . . . 9

5.2 Cross-encoders . . . 9

5.2.1 Why the accurate model cannot go first . . . 10

5.3 Re-ranking, measured . . . 10

5.4 The ceiling the re-ranker cannot pass . . . 11

6 The whole table, in one place **11**

6.1 How to choose, in practice . . . 11

7 The notebook **12**

8 Design exercise: a compliance-software company's search over its customers' policies **12**

9 Exercises **14**

10 Summary **15**

Where we left off

Lecture 19 set up the problem and its lexical answer. The task is to rank the 5,183 SciFact abstracts for each of 300 claims; a cheap first stage searches everything and is judged on recall at depth, because whatever it never returns no later stage can recover. BM25 — idf, saturation and length normalisation over an inverted index, with no training at all — scored 0.6611 NDCG@10. Rankings were measured with precision and recall at a cutoff, reciprocal rank, average precision and NDCG, each reading the order of a ranking differently. And every one of those numbers rests on relevance judgements: a person's decision on a claim–abstract pair. SciFact's come from the abstracts each claim's source sentence cited, are binary and sparse, and anything left unjudged is scored as irrelevant.

BM25 also failed in a way no parameter could fix: the *P. chabaudi* claim's one relevant abstract

shares two words with it — `in` and `to` — and BM25 ranks that abstract 4,999 of 5,183. That leaves two separate gaps, needing two separate answers. *Ranked too low*: recall@10 is 77.8% but recall@100 is 88.5%, so those documents were found and misplaced. *Never found*: the rest, which no re-ordering of what BM25 returned can reach.

1 What an index of strings cannot do

Measured on SciFact's judged pairs, a relevant abstract is missing 40.9% of its claim's words on average, and 28.6% of judged pairs share fewer than half. *Heart attack* and *myocardial infarction*; *lower numbers* and *reduced inoculum*; a mechanism described rather than named.

The requirement

A representation in which two texts that mean the same thing are *close*, whether or not they share a word. Which is precisely what Lectures 17 and 18 built — and the reason Part V comes after Part IV rather than before it.

1.1 The three repairs that do not work

Repair	Why it falls short
Stemming	merges word forms, not concepts — nothing links *infarction* to *attack*
A synonym dictionary	someone must write it, per domain, and it cannot cover paraphrase
Query expansion	adds terms from the top results, so it needs the answer to already be near the top — which is exactly what has failed

Each is a patch on the same assumption: that relevance is visible in shared strings. The assumption is what has to go. This is the shape of every move from feature engineering to learned representations in this course — the patches are not stupid, they are *bounded*, and the bound is the assumption.

2 Bi-encoders

Lecture 18 left us with one vector per token, each depending on its context. The new idea of this lecture is to put the query and the document in the same space and make the score a dot product; everything else follows from that choice.

2.1 From tokens to one vector

A retrieval score needs one vector per *text*, so the token vectors must be pooled.

Pooling	What it does
`[CLS]`	take the reserved first token's vector — meaningful only if the model was trained to put something there
Mean	average the token vectors, *masking padding*; robust, and what our model uses
Max	component-wise maximum; rarely better, and discards how often a thing was said

Failure condition — masking the padding is not optional

Mean-pooling over the padding tokens makes a text's vector depend on *the batch it was encoded in*. The same abstract gets two different vectors, the retrieval scores change, and nothing raises an error.

2.2 Why the vectors are normalised

Normalise to unit length and the inner product becomes the cosine, which discards magnitude. That is deliberate.

- Magnitude in these encoders varies with length, not relevance. Under this lecture's mean pooling, longer abstracts get shorter vectors, so an unnormalised dot product quietly prefers short documents — a length bias of the kind BM25's b was invented to correct.
- Normalised scores are bounded in $[-1, 1]$, which makes a threshold transferable between corpora.

The same trap as Lecture 19's length normalisation, arriving from a different direction. Length decides part of the ranking unless something is done about it. Here the model does it: `all-MiniLM-L6-v2` ends in a normalisation layer, so its vectors come out unit length whether or not you ask.

2.3 Encode both sides into one space

$$\text{score}(q, d) \;=\; E(q) \cdot E(d), \qquad E : \text{text} \to \mathbb{R}^{384}.$$

One encoder, applied *separately* to the query and to the document.

Why "separately" is the whole design

$E(d)$ does not depend on the query. So every document can be encoded once, offline, and *stored*; at request time you encode one short query and take 5,183 inner products. The expensive part happens before anyone asks.

Hold on to that sentence. It is also the reason the bi-encoder is the weaker of the two neural models in this lecture.

One encoder or two?

	Shared encoder	Two encoders
Parameters	one set	two sets
Suits	symmetric tasks — sentence similarity	asymmetric — short query, long document
Risk	a short query and a long abstract are not the same kind of text	twice the parameters on the same labels

We use a shared encoder because the model we borrow was trained that way. It is a real modelling choice and it should be reported: a paper that says "bi-encoder" without saying which has told you less than it thinks.

2.4 The whole retrieval system, in six lines

```
D = encode(model, documents)              # once, offline, 5,183 x 384
D = D / np.linalg.norm(D, axis=1, keepdims=True)

def search(query, k=10):
    q = encode(model, [query])[0]
    q /= np.linalg.norm(q)
    s = D @ q                             # every document, exactly
    return np.argsort(-s)[:k]
```

Note what is *not* here: no index structure, no library, no database. The matrix product is the search, and on this corpus it is fast enough to be exact. Everything a vector database adds is a way of not doing that matrix product in full, and it is bought with recall.

2.5 The dimension is a dial too

384 dimensions is a choice the model made for us, and its consequences are arithmetic:

Bytes per document	1,536
This corpus	8.0 MB
The same corpus at 768 dimensions	15.9 MB
Ten million documents at 384-d	15.4 GB

Every one of those bytes is read on every exact query. A dense index is expensive not to build but *to use*: the inverted index skips every document that shares no term with the query, and the exact dense scan cannot skip anything.

3 Training one

3.1 What the objective must say

We want $E(q) \cdot E(d^+)$ large and $E(q) \cdot E(d^-)$ small. Written as a softmax over one positive and a set of negatives:

The contrastive objective

$$\mathcal{L} \;=\; -\log \frac{\exp\big(E(q) \cdot E(d^+)/\tau\big)}{\exp\big(E(q) \cdot E(d^+)/\tau\big) + \sum_{d^-} \exp\big(E(q) \cdot E(d^-)/\tau\big)}$$

which is the cross-entropy of Lecture 17 with the *documents* as classes, and the objective you meet again in Lecture 23 with images on one side. τ is a temperature: it scales the cosines before the softmax.

The positives come from the judgements. The negatives are the problem: there are millions of documents and you cannot score them all, so a few are chosen — and the obvious way to choose them is where training goes wrong.

3.2 The false-negative problem, sized

A random abstract is a trivially easy negative, so **hard negatives** are mined instead: take a first stage's top results and call the unjudged ones negative.

Failure condition — unjudged is not irrelevant

On this corpus, of 1.55 million claim–abstract pairs only 339 are judged relevant, a mean of 1.13 per claim. BM25's top ten for a claim contains on average 0.86 judged-relevant abstracts and 9.14 others — and those others are *unjudged*, not judged irrelevant, so on a corpus this sparsely judged some of them are relevant. Train on them as negatives and the gradient pushes the model away from correct answers. This is the single commonest reason a fine-tuned retriever underperforms the paper it copied.

3.3 What the bi-encoder gave up

A single 384-dimensional vector must stand for a 225-token abstract, and it is computed *before the query is known*.

- An abstract about three things compresses to one point, so it is somewhat about each and exactly about none.
- Nothing in the encoder can emphasise the part of the document the query is asking about — it has not been asked yet.
- Exact term matching is not guaranteed: two different gene names may land close together.

That is the price of indexability, and the trade is *structural*: it is not fixed by a bigger encoder. It is the whole reason a second, non-indexable model appears later in this lecture.

3.4 The operational cost the accuracy table never shows

	Inverted index	Dense index
New document	append to its terms' postings; microseconds	one encoder pass, then append the vector
New model version	nothing to do	re-encode the entire corpus
Mixed versions	impossible	silently wrong — vectors from two models are not comparable

The last row is a real outage, and a quiet one: nothing errors, the scores are simply meaningless for the documents encoded by the old model. A dense index needs a model version stored beside it.

4 Against BM25, on the same judgements

4.1 The protocol, stated before the numbers

Same corpus of 5,183 abstracts, same 300 test claims, same judgements. The dense model is `all-MiniLM-L6-v2`, trained on general web sentence pairs and never shown a scientific claim, and it is used **zero shot**: no fine-tuning on SciFact. Same metric code — the functions written in Lecture 19's notebook, unchanged. Both retrievers scored *exactly*: every document for every query, no approximate index in either. And the metric was named before the run: NDCG@10, with recall@100 reported alongside because that is what a first stage is judged on.

4.2 The result

Metric	BM25	Dense, zero shot
NDCG@10	0.6611	0.6451
Precision@1	0.5433	0.5033
MRR	0.6350	0.6113
Recall@10	0.7784	0.7833
Recall@100	0.8852	0.9250

The neural model loses on the headline metric

Thirty years of age, no training, no accelerator, no vector store — and BM25 is ahead by 0.0160 NDCG@10.

What this result does not license

- It does *not* say dense retrieval is worse in general. Fine-tuned on in-domain pairs it beats BM25 on this corpus, and that is well replicated.
- It does *not* say embeddings are a bad idea. Recall@100 went up, and the mismatch case went from rank 4,999 to rank 8.

- It *does* say that this particular substitution — off-the-shelf encoder for BM25, no labels, technical corpus — is a loss, and that is a decision many teams make without measuring.

Stating the scope of a claim is not hedging. A result whose boundary you cannot draw is a result you cannot use.

4.3 But look at recall@100

Metric	BM25	Dense	Difference
NDCG@10	0.6611	0.6451	−0.0160
Recall@10	0.7784	0.7833	+0.0049
Recall@100	0.8852	0.9250	+0.0398

The dense retriever is *worse at ordering and better at finding*. Recall from Lecture 19 what a first stage is judged on: not what the user sees, but what it makes possible. On that criterion — the criterion for the job it would actually be given — the dense retriever wins.

4.4 Do they find the same documents?

Take every relevant abstract in the test set and ask which method placed it in the top ten.

Found by	Count	Share
Both	227	67.0%
BM25 only	30	8.8%
Dense only	38	11.2%
Neither	44	13.0%

68 of 339 relevant abstracts are found by exactly one of the two. They are not two attempts at the same ranking; they fail on *different documents*.

Failure condition — a mean over queries

BM25 0.6611 against dense 0.6451 sounds like two systems doing almost the same thing. Per claim they frequently disagree completely. Two systems with the same mean can have entirely different error sets: the mean answers "which is better on average" and cannot answer "are these the same system" — and only the second question tells you whether fusing them is worth anything.

4.5 The claim BM25 could not find

"In mice, P. chabaudi parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers."

Rank of its one relevant abstract	
BM25	4,999
Dense, zero shot	8

Nothing about the text changed; the abstract still shares almost no words with the claim. What changed is that the comparison is no longer between strings. Be precise about what this shows: not that the dense retriever is better — the table above says it is not — but that the two methods fail *independently*.

5 Combining them, and paying for it

5.1 Hybrid retrieval

The scores are not on a comparable scale — BM25 returns unbounded sums, the dense model returns cosines — so fuse the *ranks* instead:

Reciprocal rank fusion

$$\mathrm{RRF}(d) \;=\; \sum_{\text{systems}} \frac{1}{k + \mathrm{rank}(d)}, \qquad k = 60.$$

No training, no tuning, no calibration, and it needs nothing from the two systems but their orderings — which is why it is what people actually deploy. The constant k damps the top ranks so one system cannot dominate on a single confident hit; 60 is conventional, in the same sense that $\log_2(i+1)$ is.

Metric	BM25	Dense	Hybrid
NDCG@10	0.6611	0.6451	0.6948
Precision@1	0.5433	0.5033	0.5633
Recall@10	0.7784	0.7833	0.8239
Recall@100	0.8852	0.9250	0.9617

Better than either, on every metric, from adding two reciprocals. The gain over BM25 is +0.0338 NDCG@10 — larger than anything else in this lecture will buy you. And it is not mysterious: 68 relevant abstracts were found by exactly one system, and fusion is how you keep both.

5.2 Cross-encoders

$$\mathrm{score}(q, d) = \mathrm{Transformer}\big(\,\texttt{[CLS]}\ q\ \texttt{[SEP]}\ d\,\big)$$

Concatenate the query and the document and put both through the model *together*. Every token of the query can attend to every token of the document, which is exactly what a bi-encoder forbids by construction.

	Bi-encoder	Cross-encoder
Sees	each text alone	the pair, jointly
Document vectors	precomputed	impossible — they depend on the query
Cost per query	one encoding + N dot products	N transformer passes
Accuracy	lower	higher

One sentence worth being able to say

The accuracy difference and the cost difference have the *same cause*: whether the query is present when the document is read.

Why the accurate model cannot go first

A full cross-encoder scan of this corpus, for these claims, is

$$5{,}183 \times 300 = 1{,}554{,}900 \text{ transformer passes}$$

on the toy corpus of this course. A web index is 10^{10} documents and the user is waiting 200 ms, so the arithmetic is not close — it is off by ten orders of magnitude. Hence the pipeline: a cheap first stage reduces millions to a hundred, and an expensive second stage reorders that hundred. The architecture of every production search system is a direct consequence of the fact that the good model does not scale.

Failure condition — an accurate model that cannot be indexed

The cross-encoder reads query and document together, which is exactly why it is stronger and why it cannot be precomputed. Put it first and the latency is the corpus size; the quality number says nothing about whether the system can be run.

Stage	Model passes per query
BM25	0
Bi-encoder	1 (the query)
Cross-encoder over top 10	10
Cross-encoder over top 100	100
Cross-encoder over the corpus	5,183

The last row is why the pipeline exists, and the middle rows are why its depth is the parameter you actually argue about in a design review. Both are the same arithmetic.

5.3 Re-ranking, measured

Cross-encoder over BM25's top k, everything below k left where it was:

System	NDCG@10	P@1	Pairs scored
BM25 alone	0.6611	0.5433	0
+ re-rank top 10	0.6656	0.5600	3,000
+ re-rank top 50	0.6773	0.5667	15,000
+ re-rank top 100	0.6781	0.5667	30,000

Going from 10 to 50 buys +0.0117; going from 50 to 100 buys +0.0008 for twice the compute. The first ten candidates give most of the gain and the next ninety give a tenth of it. That is the shape of every re-ranking curve you will meet, and the reason production depths are usually in the tens rather than the thousands.

Note also that recall@10 barely moves: re-ranking the top 10 cannot change *which* documents are in the top 10. It changes their order, and P@1 with it.

5.4 The ceiling the re-ranker cannot pass

Re-ranking reorders; it cannot retrieve. So the recall of the first stage at depth k is a hard bound on the second stage at depth k:

Depth k	10	50	100	1000
BM25 recall@k	0.7784	0.8654	0.8852	0.9650

Re-rank BM25's top 100 and a *perfect* cross-encoder still cannot exceed recall 0.8852 at any cutoff — 11.5% of relevant abstracts are simply not in the candidate set. Notice that the hybrid first stage reaches recall@100 of 0.9617.

The most useful sentence in this lecture

Improving the first stage raises the ceiling for everything above it. Improving the second stage never does.

6 The whole table, in one place

System	**NDCG@10**	**R@100**	**Cost per query**
Corpus unranked	0.0012	0.0117	nothing
BM25	0.6611	0.8852	postings walk
Dense, zero shot	0.6451	0.9250	1 encode + full scan
BM25 + cross-encoder@100	0.6781	0.8852	+ 100 passes
Hybrid (BM25 + dense, RRF)	0.6948	0.9617	1 encode + both

The best NDCG@10 in this lecture comes from the row with *no cross-encoder in it* — two first-stage retrievers, used as downloaded, and a sum of reciprocals that learns nothing, ahead of a hundred transformer passes per claim.

6.1 How to choose, in practice

Situation	**What to do**
No labels, technical vocabulary	BM25. Measure before considering anything else
No labels, everyday language, paraphrase matters	hybrid — BM25 plus an off-the-shelf encoder, fused
Thousands of labelled pairs in your domain	fine-tune a bi-encoder; now it can beat BM25
Latency budget allows a second stage	add a cross-encoder over the top 50–100

Every row starts with BM25 in the table. Not because it wins — on your corpus it may not — but because a comparison without it is not a measurement.

7 The notebook

The notebook runs on free CPU; encoding the corpus once is most of the work, and nothing is trained. It uses the same BM25 code as Lecture 19, so the baseline in the comparison is the baseline you already built, and it scores the zero-shot bi-encoder exactly, counts which relevant abstracts each system finds, and writes RRF and cross-encoder re-ranking at three depths out by hand — no vector database.

What to change

Re-rank the *hybrid* top 100 rather than BM25's. The ceiling argument predicts what should happen to recall; check whether NDCG follows, and say why the two need not move together.

8 Design exercise: a compliance-software company's search over its customers' policies

The lecture closes with a design exercise in the manner of an ML system-design interview, on a product with the shape of data this model suits.

The task. A company sells a compliance platform to banks, insurers and pharmaceutical firms, each of which loads its own policies into it. The product team wants a search box: an employee types "can I accept a gift from a supplier?" and gets, in a fraction of a second, a ranked list of clauses from their employer's policies, the one that answers the question at the top. The policy says "hospitality from third parties may be declined"; the employee never uses those words, and that mismatch is the whole problem. The business measures renewals, the support tickets the box removes, and the cost of an employee acting against a policy nobody found. Given: every customer's policy library — a few hundred documents each, in numbered clauses — and the question log of the old keyword search, a sample of it judged by compliance officers at a handful of pilot customers. Tens of thousands of clauses per customer, hundreds of customers, a few thousand questions a day. The output feeds the search box, the customers' compliance officers, and the audit trail a regulator can ask for.

1. **Frame it.** The output is a ranking of clauses, so the metrics are the ranking metrics: NDCG at ten for what the employee sees, recall at a hundred for the first stage, because whatever it never returns no later stage can recover. Both are named before the first run, the cut-off fixed to the ten results the product shows, and the baseline is the keyword search the customers already have — idf, saturation and length normalisation over an inverted index, no training — on the same clauses and judgements. *Pitfall:* naming the metric after seeing the scores. An off-the-shelf encoder that loses on NDCG at ten and wins on recall at a hundred invites a sales deck that quotes the second number, and a customer that renews on the claim and sees worse results churns at the next renewal; state the metric first, report both, and never leave the keyword baseline out of the table.

2. **Chunking.** The encoder produces one vector per text, computed before any question is asked, and a whole policy is about everything — gifts, travel, data, conflicts of interest — so its one vector is somewhat about each topic and exactly about none. Cut at the document's own joints — one passage per clause, carrying the policy's title and the section's heading — and

treat the passage size as a dial read on recall at a hundred. *Pitfall:* what a chunk loses. A clause that says "the limit in the preceding clause does not apply to gifts from regulators" is unanswerable alone, and one cut mid-sentence retrieves for neither half; an employee who reads the limit without its exception acts on it, and the customer explains the breach to its regulator. Keep the cross-reference's target in the chunk, split only at the structure, and store each passage's clause identifier so that the whole clause is shown once the passage is found.

3. **The judgements, and the split.** A judgement is a person's decision on a (question, clause) pair: binary, sparse, and anything unjudged is scored as irrelevant. Split the judged questions into a tuning set, on which every choice is made, and a test set read once before the release decision. *Pitfall:* unjudged is not irrelevant. If the officers judge only what the old keyword search returned, every clause the dense model finds and the old search missed is scored as a miss, and the product team cancels the feature on a number that measured the old system. Pool the candidates of both systems before anyone judges, and re-pool when a system changes.

4. **The bi-encoder and its index.** One encoder applied separately to question and clause; mean pooling over the real tokens, masking the padding; vectors normalised, so that a long clause is not quietly penalised. The index is the matrix of clause vectors, encoded once — one matrix per customer, so that no search ever crosses a tenant; on tens of thousands of clauses the exact matrix product is the search, with nothing bought with recall. Start zero shot, and fine-tune once thousands of judged pairs exist. *Pitfall:* an encoder trained on web sentence pairs that never saw a policy can lose to the keyword search on the headline metric while winning on recall at depth — measure it before the data team promises a date; and when fine-tuning, do not mine hard negatives from the unjudged top of the list, because on a sparsely judged corpus some of them answer the question and the gradient pushes the model away from correct answers.

5. **Where dense fails, and the hybrid.** Many questions name a thing exactly — a policy code, a regulation's article, a product name, a jurisdiction — and exact matching is not guaranteed in the vector space, while the keyword search gets these right. The two systems fail on different questions, which is the signal that fusing helps: count per judged question which system found the answer, and fuse the *ranks* by reciprocal rank fusion. *Pitfall:* adding the scores. A lexical score is an unbounded sum and a cosine is bounded in a narrow band, so one system dominates whenever it is confident, which on exact identifiers is always — and a compliance officer who searches a policy by its code and does not find it files a support ticket. Fuse ranks, and keep the overlap table in the release review.

6. **Re-ranking.** A cross-encoder reads question and clause together and is stronger for exactly the reason it cannot be precomputed, so it runs second, over the fused top of the list, at a depth chosen on the tuning questions against the latency the product's service agreement allows; the first tens of candidates buy most of the gain. *Pitfall:* putting the accurate model first — tens of thousands of transformer passes per question, and the quality number says nothing about whether the system can be run within the agreement or what it costs per query. Re-ranking reorders what it is given; it never changes which clauses are in the candidate set.

7. **Evaluate, and read the ceiling.** One table on the test questions — keyword, dense, hybrid, hybrid re-ranked — with NDCG at ten and recall at a hundred side by side, read once, and the misses inspected per customer, because a mean over questions hides which ones and

which tenant. The recall of the first stage at depth k is a hard bound on everything above it. *Pitfall:* spending the next quarter on a bigger re-ranker while the first stage misses one judged answer in ten. Improving the second stage never raises the ceiling; improving the first stage — hybrid, better chunking, a fine-tuned encoder — raises it for everything above. Read recall at depth before the roadmap is written.

8. **In production.** Customers edit policies daily: a new clause is one encoder pass and an appended vector, a withdrawn one a vector removed, with the inverted index updated in the same step. A new encoder is a re-encoding of every customer's corpus, built beside the old index, with the judged questions re-run as the regression test, released to a cohort of customers first and read on clicks on the top result against the old system; the questions that end without a click are monitored per customer, and the on-call engineer owns the index, not only the service. *Pitfall:* mixed versions. Clauses encoded by two versions of the model in one customer's index raise no error — the older clauses' scores are simply meaningless, the policy amended last quarter is never found again, and an employee acts on the withdrawn rule. Store the model version beside each index, refuse to search a mismatched one, and switch once, never in place.

9. **Ship it.** Each customer's policies → passages cut at clauses, headings attached → judged questions pooled from both systems, split into tuning and test → a keyword index and normalised clause vectors per customer, one model version → ranks fused → cross-encoder over the top → NDCG at ten and recall at a hundred, read once → cohort release, rebuild on change, re-run the judged set, monitor.

9 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 21's deck and in the exercise book.

1. Explain why a bi-encoder can be indexed and a cross-encoder cannot, and say what the two differences have in common as a cause. [5]

2. A first stage has recall@100 of 0.885. State the highest recall a perfect re-ranker over those 100 candidates can reach, and what follows for where to spend effort. [4]

3. BM25 scores 0.6611 NDCG@10 on SciFact and a zero-shot bi-encoder 0.6451, yet fusing their rankings by reciprocal rank fusion scores 0.6948. State the measurement that predicts the gain, and why the fusion uses ranks rather than scores. [5]

4. Hard negatives are mined by calling the unjudged documents in a first stage's top results negative. On SciFact, BM25's top 10 holds on average 0.86 judged-relevant abstracts and 9.14 others. State the trap, and what it does to training. [5]

5. Zero shot on SciFact, a bi-encoder loses to BM25 on NDCG@10 (0.6451 against 0.6611) and wins on recall@100 (0.9250 against 0.8852). Say which job each metric judges, and which system you would put in the first stage. [4]

10 Summary

A lexical index matches strings and relevance is about meaning, and no patch on the index closes that gap. A bi-encoder can be indexed because $E(d)$ does not depend on the query — and that same independence is why it is the weaker model. Zero-shot on this corpus it loses to BM25 on NDCG@10, 0.6451 against 0.6611, and wins on recall@100, 0.9250 against 0.8852. The two fail on different documents: 68 of 339 relevant abstracts were found by exactly one, so fusing them beats both at 0.6948. And the accurate model goes second because it does not scale — and it can never exceed the first stage's recall at that depth. The closing design exercise built a compliance-software company's search over its customers' policies in nine steps — both metrics named first with the keyword baseline in the table, passages cut at the clauses, judgements pooled from both systems, a normalised dense index per customer measured zero shot, ranks fused for the policy codes dense loses, a cross-encoder over the top within the service agreement, recall at depth read as the ceiling, and one model version per index — each step named for the pitfall it avoids.